

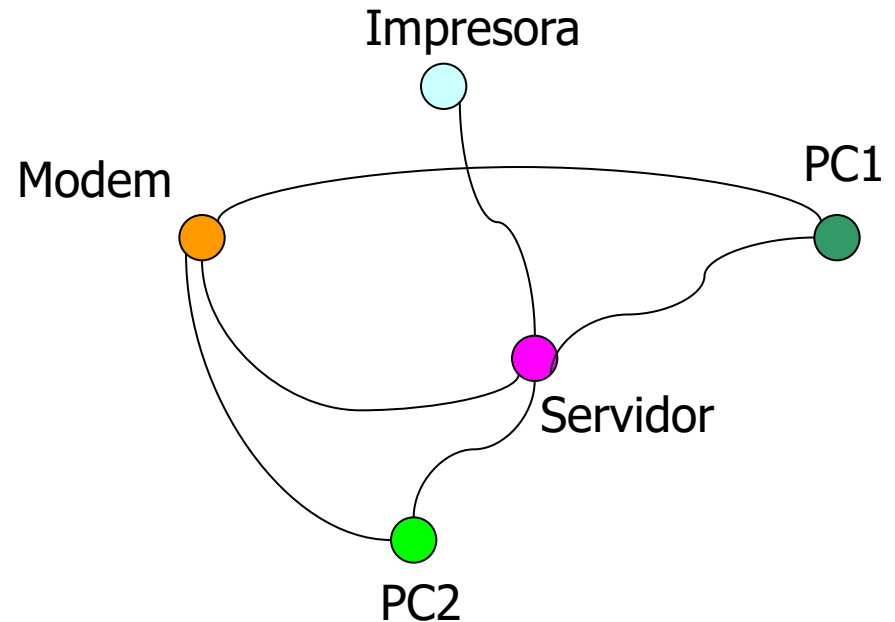
Estructuras de Datos

Grafos

INTRODUCCION

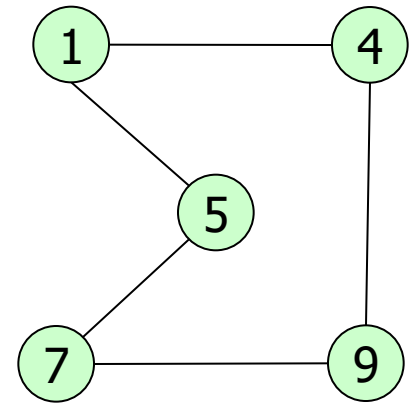
- ▣ Los grafos son estructuras de datos
- ▣ Representan relaciones entre objetos
 - ▣ Relaciones arbitrarias, es decir
 - ▣ No jerárquicas
- ▣ Son aplicables en
 - ▣ Química
 - ▣ Geografía
 - ▣ Ing. Eléctrica e Industrial, etc.
 - ▣ Modelado de Redes
 - ▣ *De alcantarillado*
 - ▣ *Eléctricas*
 - ▣ *Etc.*

Dado un escenario donde ciertos objetos se relacionan, se puede "modela el grafo" y luego aplicar algoritmos para resolver diversos problemas



DEFINICION

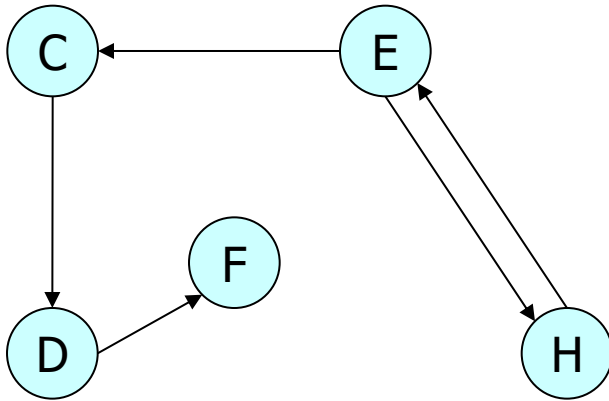
- Un grafo $G = (V, A)$
- V , el conjunto de vértices o nodos
 - Representan los objetos
- A , el conjunto de arcos
 - Representan las relaciones



$V = \{1, 4, 5, 7, 9\}$

$A = \{(1,4), (5,1), (7,9), (7,5), (4,9), (4,1), (1,5), (9,7), (5, 7), (9,4)\}$

TIPOS DE GRAFOS



$V = \{C, D, E, F, H\}$

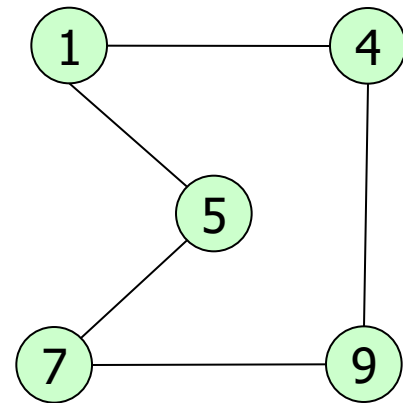
$A = \{(C,D), (D,F), (E,H), (H,E), (E,C)\}$

▣ Grafos no dirigidos

- ▣ Si los pares de nodos de los arcos
- ▣ No son ordenados Ej.: u-v

▣ Grafos dirigidos

- ▣ Si los pares de nodos que forman arcos
- ▣ Son ordenados. Ej.: (u->v)



Grafo del ejemplo anterior

OTROS CONCEPTOS

▣ Arista

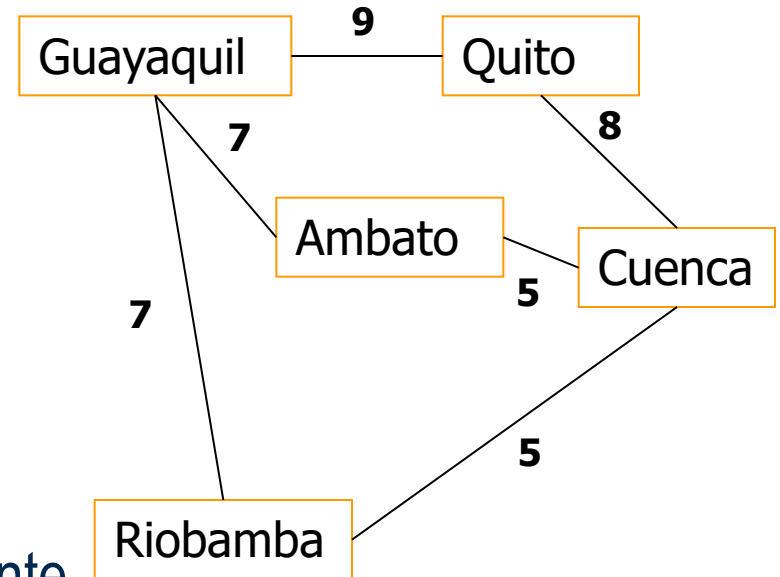
- ▣ Es un arco de un grafo no dirigido

▣ Vertices adyacente

- ▣ Vertices unidos por un arco

▣ Factor de Peso

- ▣ Valor que se puede asociar con un arco
- ▣ Depende de lo que el grafo represente
- ▣ Si los arcos de un grafo tienen F.P.
 - ▣ *Grafo valorado*



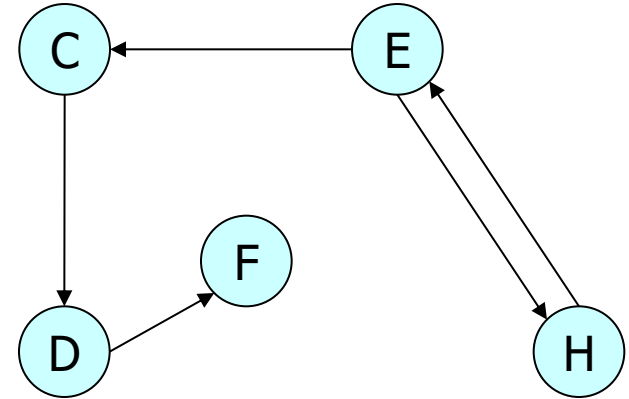
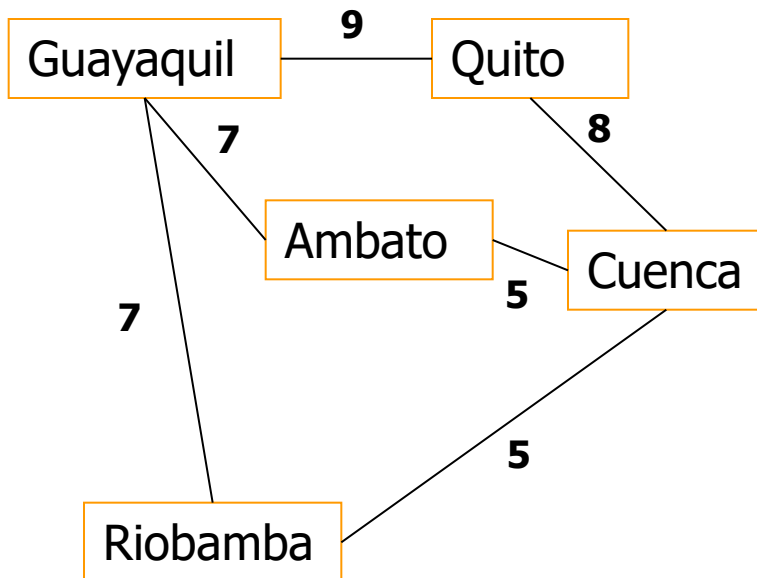
GRADOS DE UN NODO

□ En Grafo No Dirigido

□ Grado(V)

□ *Numero de aristas que contiene a V*

$$\text{Grado}(\text{Guayaquil}) = 3$$



$$\text{Gradoent}(\text{D}) = 1 \text{ y } \text{Gradsal}(\text{D}) = 1$$

□ En Grafo Dirigido

□ Grado de entrada, Graden(V)

□ *Numero de arcos que llegan a V*

□ Grado de Salida, Gradsal(V)

□ *Numero de arcos que salen de V*

CAMINOS

Definicion

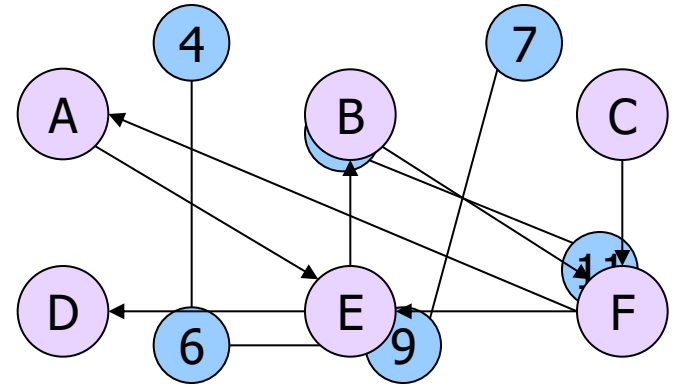
- Un camino P en un grafo G , desde V_0 a V_n
- Es la secuencia de $n+1$ vertices
- Tal que $(V_i, V_{i+1}) \in A$ para $0 \leq i \leq n$

Longitud de camino

- El numero de arcos que lo forman

Camino Simple

- Todos los nodos que lo forman son distintos



Camino AntrA 4 y 7

$P \equiv \{A, E, B, F, A\}$

Longitud: 4 – 4ciclo

Ciclo

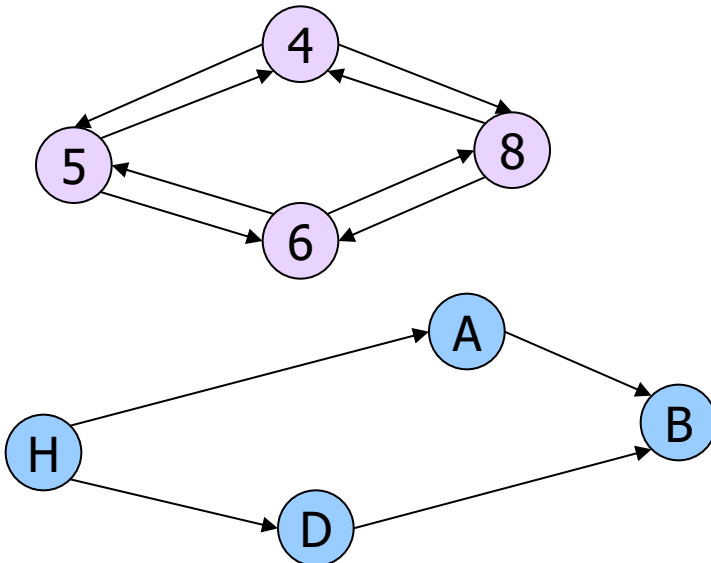
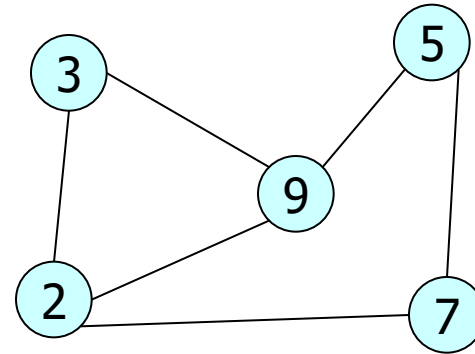
- Camino simple cerrado de long. ≥ 2
- Donde $V_0 = V_n$

CONECTIVIDAD

□ Grafo No Dirigido

□ Conexo

- *Existe un camino entre cualquier par de nodos*



□ Grafo Dirigido

□ Fuertemente Conexo

- *Existe un camino entre cualquier par de nodos*

□ Conexo

- *Existe una cadena entre cualquier par de nodos*

TDA GRAFO

□ Datos

- Vertices y
- Arcos(relacion entre vertices)

□ Operaciones

- void AñadirVertice(Grafo G, Vertice V)
 - *Añadir un nuevo vertice*
- void BorrarVertice(Grafo G, Generico clave)
 - *Eliminar un vertice existente*
- void Union(Grafo G, Vertice V1, Vertice V2)
 - *Unir dos vertices*
- Void BorrarArco(Grafo G, Vertice V1, Vertice V2)
 - *Eliminar un Arco*
- bool EsAdyacente(Grafo G, Vertice V1, Vertice V2)
 - *Conocer si dos vertices son o no adyacentes*

REPRESENTACION

□ Dos posibles representaciones

□ Estatica: Matriz de Adyacencia

- *Los vertices se representan por indices(0...n)*

- *Las relaciones de los vertices se almacenan en una Matriz*

□ Dinamica: Lista de Adyacencia

- *Los vertices forman una lista*

- *Cada vertice tiene una lista para representar sus relaciones(arcos)*

Si el grafo fuese valorado, en vez de 1, se coloca el factor de peso

MATRIZ DE ADYACENCIA

- Dado un Grafo $G = (V, A)$
- Sean los Vertices $V = \{V_0, V_1, \dots, V_n\}$

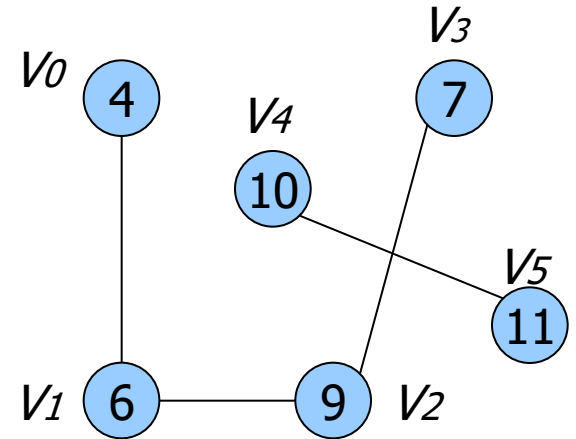
- Se pueden representar por ordinales $0, 1, \dots, n$

- Como representar los Arcos?

- Estos son enlaces entre vertices

- Puede usarse una matriz

$$a_{ij} = \begin{cases} 1, & \text{si hay arco } (V_i, V_j) \\ 0, & \text{si no hay arco } (V_i, V_j) \end{cases}$$



	V_0	V_1	V_2	V_3	V_4	V_5
V_0	0	1	0	0	0	0
V_1	1	0	1	0	0	0
V_2	0	1	0	1	0	0
V_3	0	0	1	0	0	0
V_4	0	0	0	0	0	1
V_5	0	0	0	0	1	0

EL TIPO DE DATO

□ Los Vertices

- Se definen en un Arreglo

□ Los Arcos

- Se definen en una Matriz

```
public class Grafo{  
    int MAX = 20;  
    int [][] matrizAdy;  
    Object[] vertices;  
}
```

UNIR VERTICE

```
void Union(Grafo G, int v1, int v2){  
    G.matrizAdy[v1][v2] = 1;  
    if(!G.getDirigido())  
        G.matrizAdy[v2][v1] = 1;  
}
```

LISTA DE ADYACENCIA

□ Si una matriz

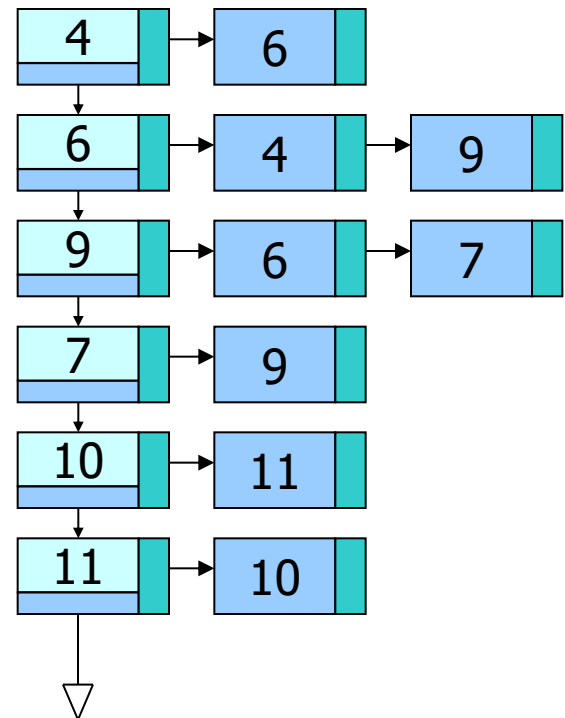
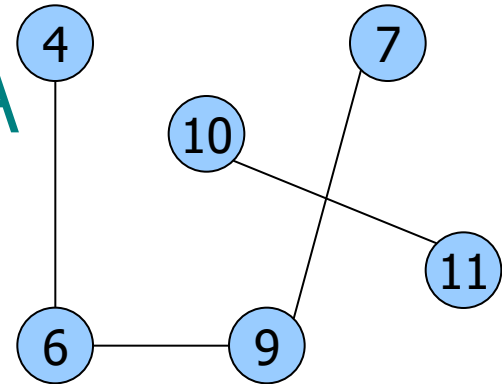
- Tiene muchos vertices y
- Pocos arcos
- La Matriz de Adyacencia
 - *Tendra demasiados ceros*
 - *Ocupara mucho espacio*

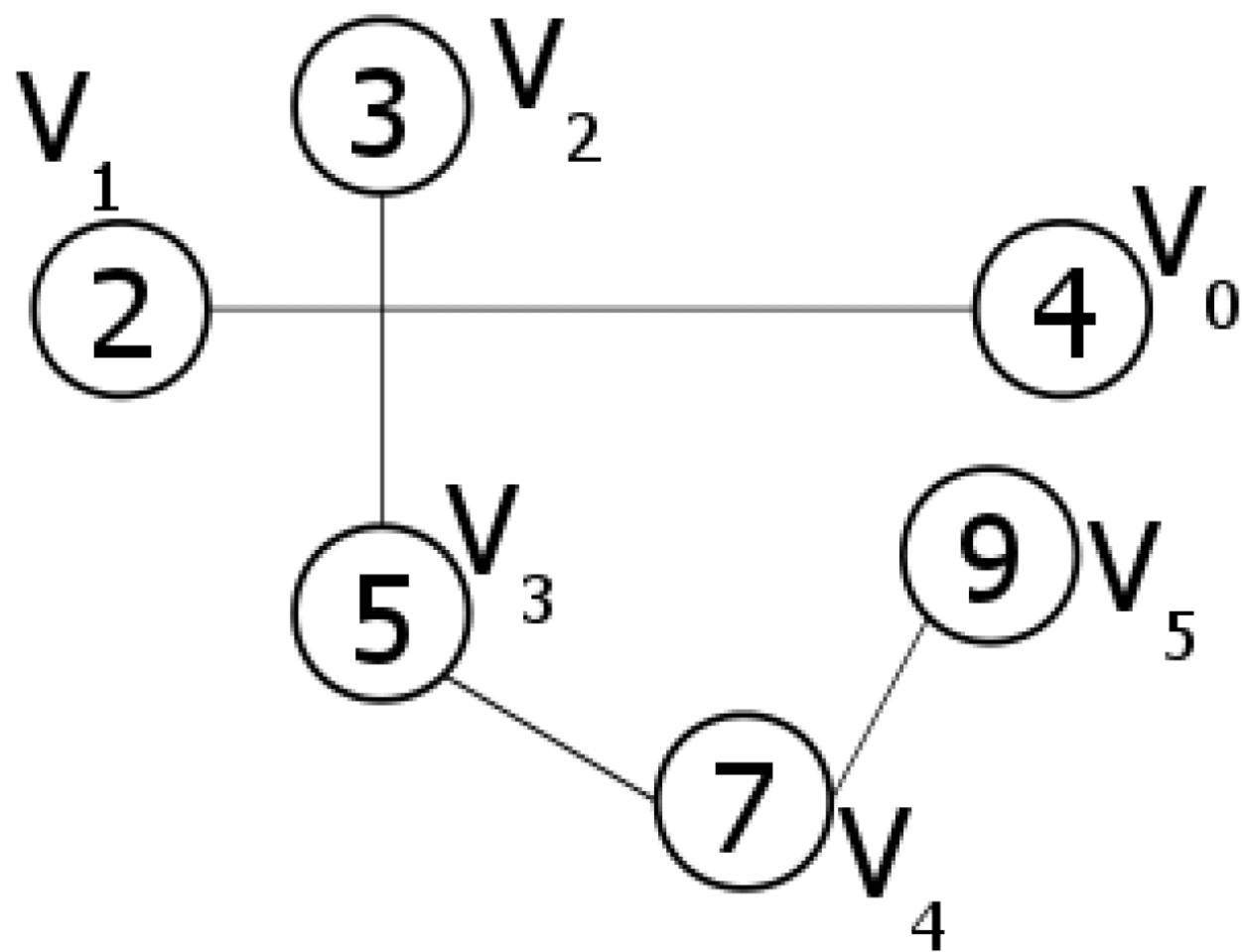
□ Los vertices

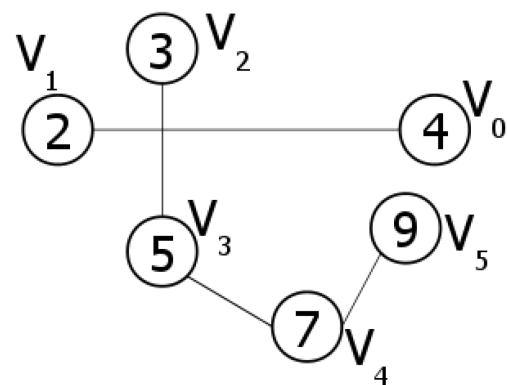
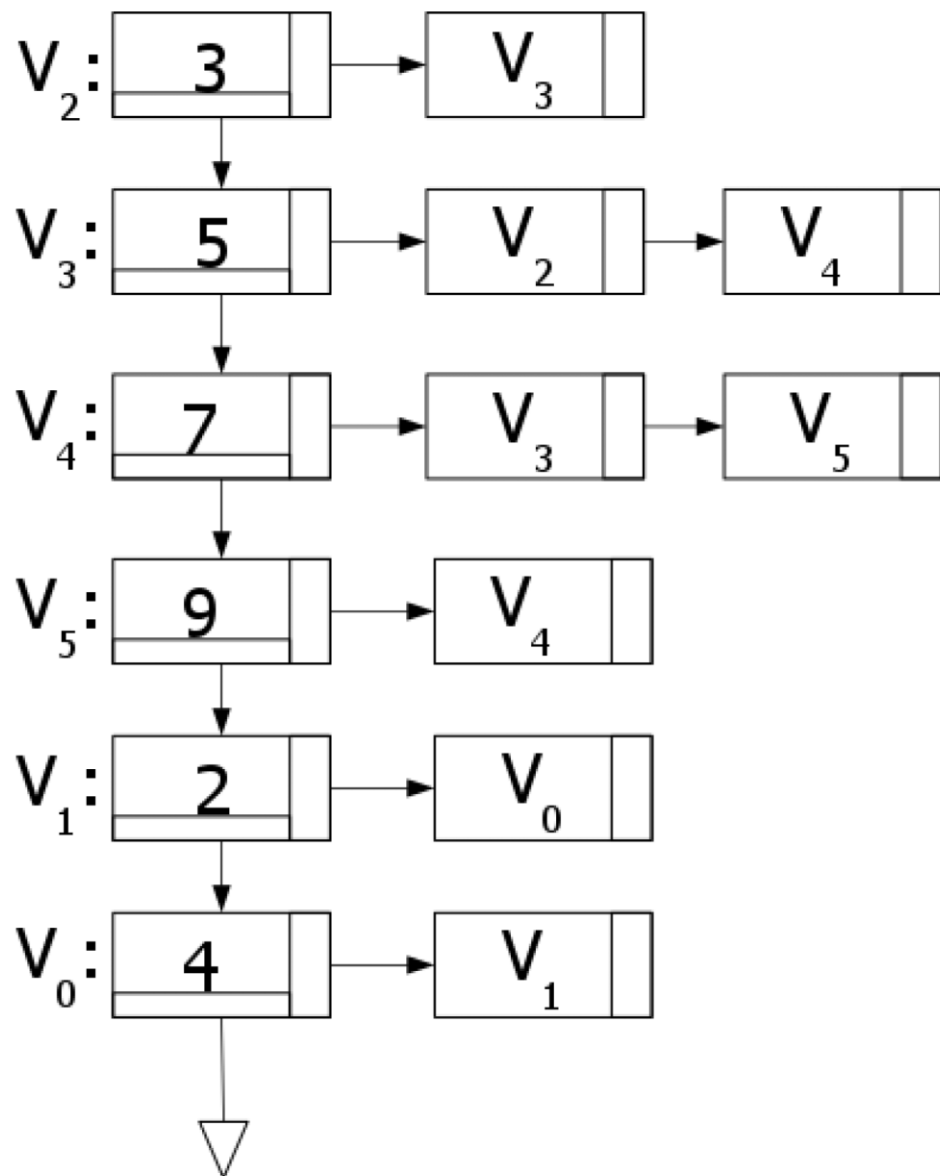
- Pueden formar una lista, no un arreglo

□ Los arcos

- Son relaciones entre vertices
- Se pueden representar con una lista x cada vertice







EL TIPO DE DATO

- Cada vertice tiene
 - Contenido
 - Una lista de adyacencia
- Cada nodo en la lista de adyacencia
 - Peso del arco
 - Siguiente
 - Una referencia al vertice(arco)

EL TIPO DE DATO

- Cada vertice tiene
 - Contenido
 - Una lista de adyacencia
- Cada nodo en la lista de adyacencia
 - Peso del arco
 - Una referencia al vertice(arco)

```
public class Vertex<V, E> {  
    V content;  
    List<Edge<E, V>> edges;  
}  
  
public class Edge<E, V> {  
    E data;  
    Vertex<V, E> source;  
    Vertex<V, E> target;  
    int weight;  
}  
  
public class Graph<V, E> {  
    List<Vertex<V, E>> vertices;  
    boolean isDirected;  
}
```

EJERCICIOS

- Complete la implementación de las operaciones del grafo con lista de adyacencia
 - Añadir vértice
 - Unir vértices (o añadir arco, conectar vértices)
 - Borrar vértice y borrar arco
 - esAdyacente

Clase Grafo

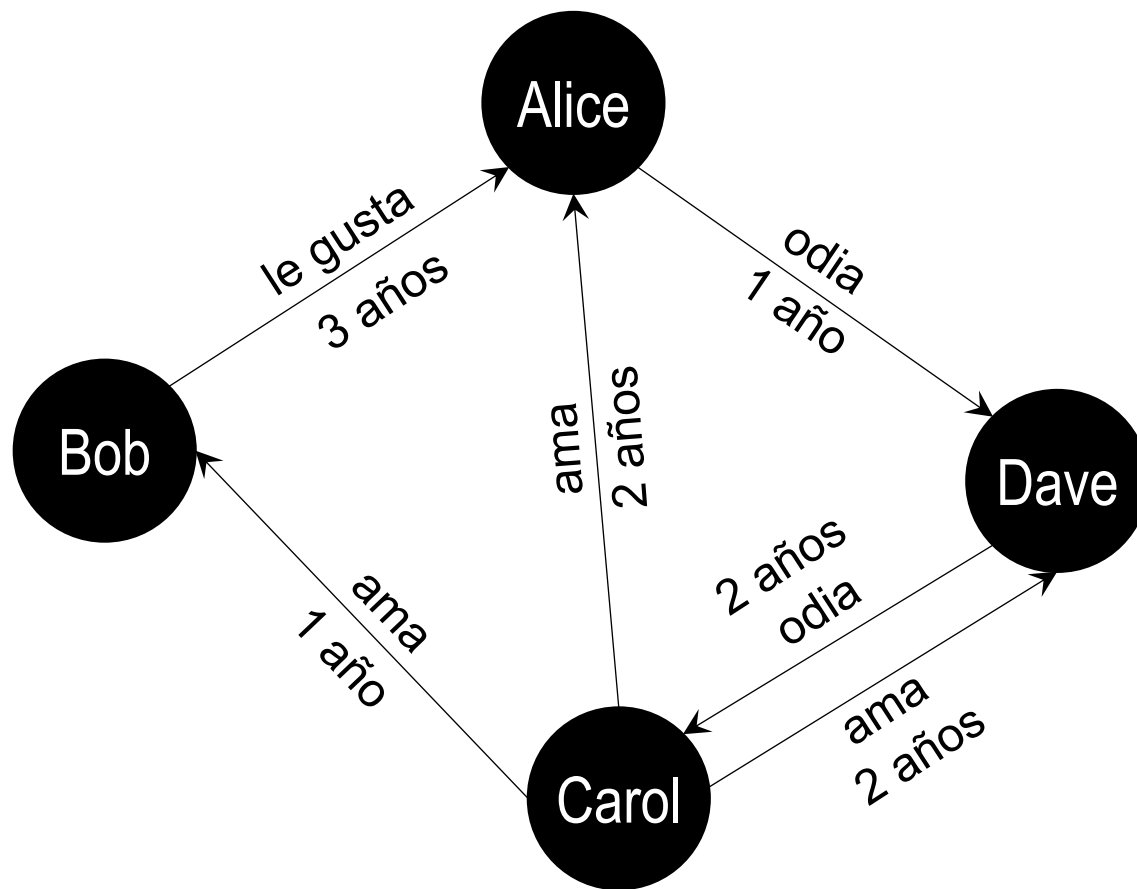
```
public class Vertex<V, E> {  
    V content;  
    List<Edge<E, V>> edges;  
    boolean isVisited;  
    int tmpDistance;  
    Vertex<V, E> predecesor;  
}
```

```
public class Edge<E, V> {  
    E data;  
    Vertex <V, E> source;  
    Vertex<V, E> target;  
    int weight;  
}
```

```
public class Graph<V, E> {  
    List< Vertex <V, E>> vertices;  
    boolean isDirected;
```

EJERCICIO

Utilizando las clases Graph, Node y Edge de la diapositiva anterior, escriba el código necesario para crear el siguiente grafo:



Nombre	Edad	Profesión	Ciudad
Alice	32	Ingeniero	Guayaquil
Bob	28	Chef	Guayaquil
Carol	27	Contadora	Quito
Dave	31	Investigador	Cuenca

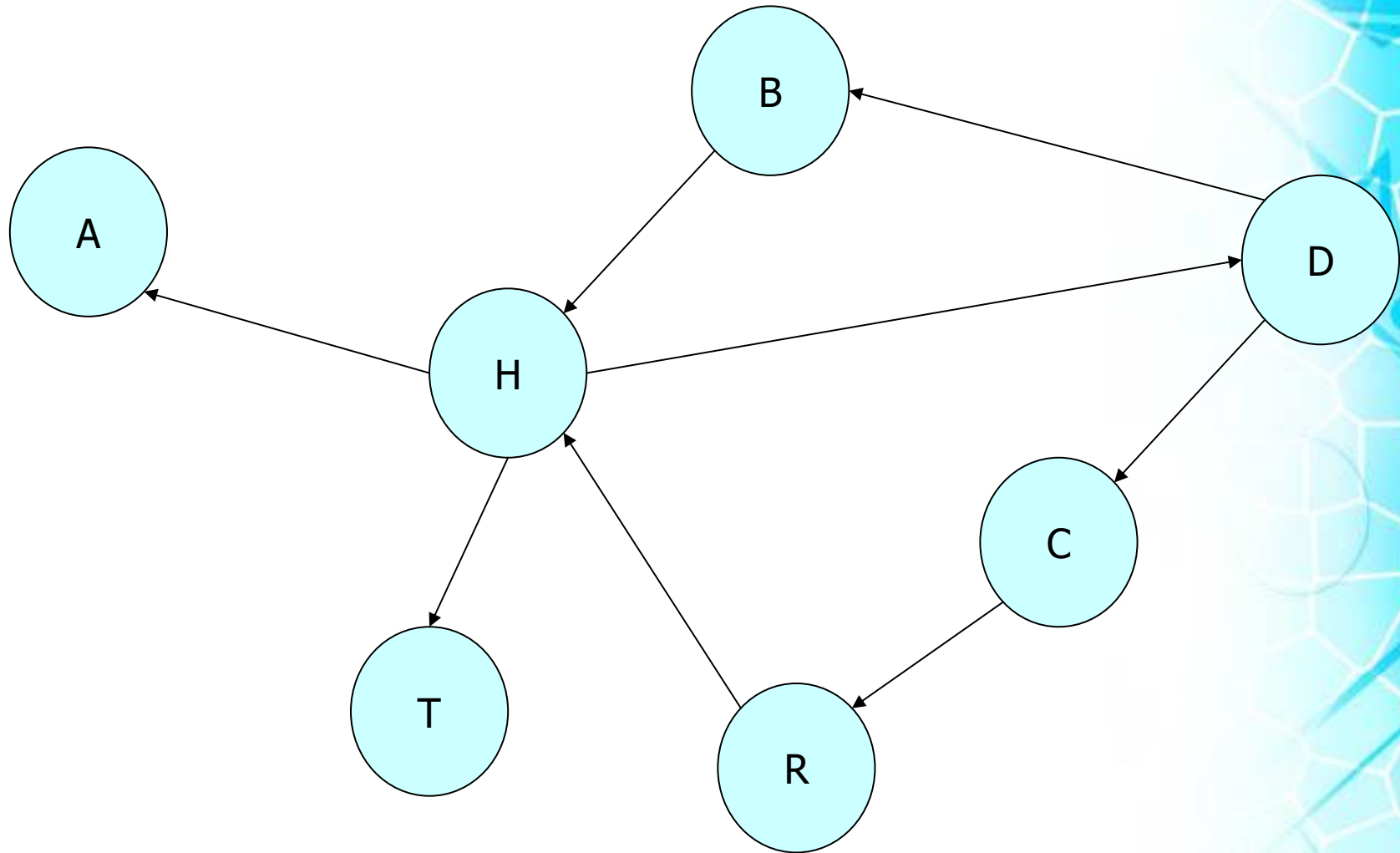
Recorridos del Grafo

RECORRIDOS DEL GRAFO

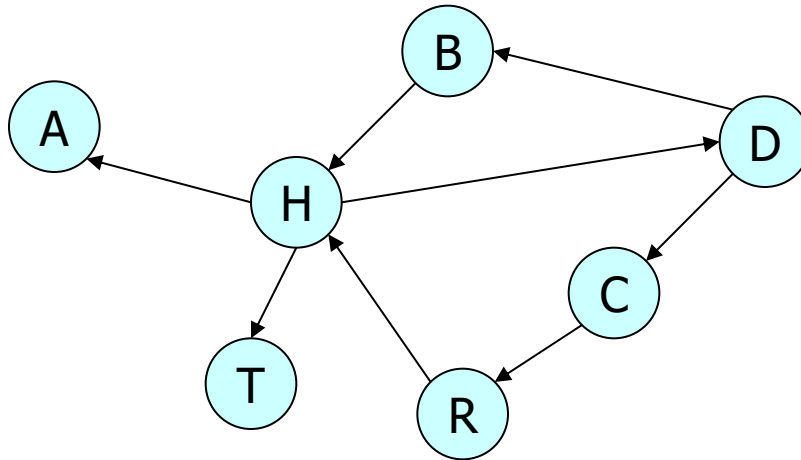
- Se desea:
 - Visitar todos los nodos **posibles**
 - Desde un vertice de partida D cualquiera
- Existe dos posibles recorridos
 - En Anchura y
 - En Profundidad

RECORRIDO EN ANCHURA

- **Encolar** vertice de partida
- Marcarlo como “visitado”
- Mientras la cola no este vacia
 - Desencolar vertice W
 - Mostrarlo
 - Marcar como visitados
 - *Los vertices adyacentes de W*
 - *Que no hayan sido ya visitados*
 - Encolarlos



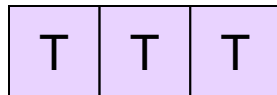
EJEMPLO



Se Muestra:

D B C H R A T

Cola

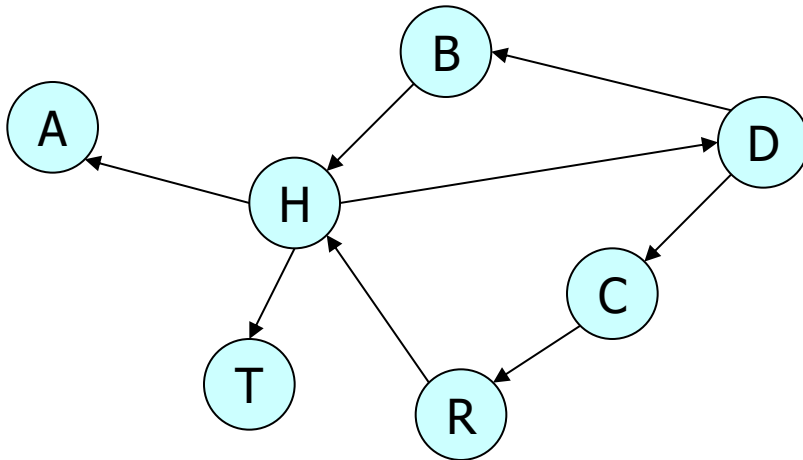


EJERCICIO

Escriba la implementación del **recorrido en anchura** de un grafo a partir de un vértice inicial

RECORRIDO EN PROFUNDIDAD

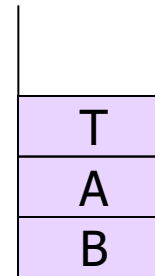
- ❑ Marcar vertice origen V como visitado
- ❑ Recorrer en Profundidad
 - ❑ Cada vertice adyacente de V
 - ❑ Que no haya sido visitado
- ❑ Ejemplo



Se Muestra:

D C R H T A B

Pila



EJERCICIO

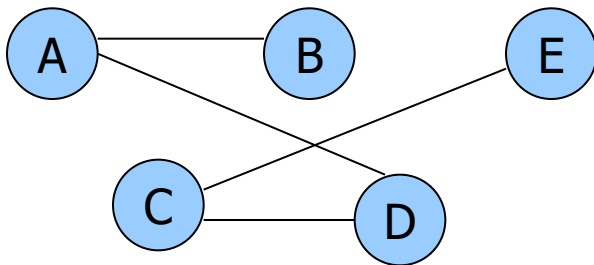
Escriba la implementación del **recorrido en profundidad** de un grafo a partir de un vértice inicial

Componentes Conexas

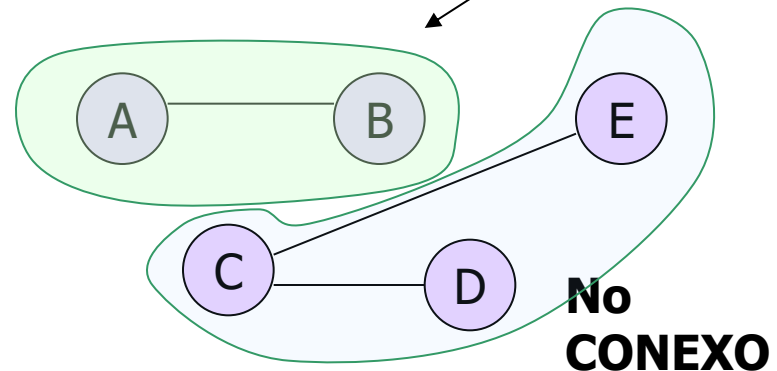
COMPONENTES CONEXAS

- De un grafo no dirigido
 - Se puede saber si es conexo
- Si no lo es se pueden conocer sus
 - Componentes Conexas
 - Conjunto W , de vértices del grafo
 - En el cual hay camino desde cualquier $V1$ a cualquier $V2$
 - Donde $V1$ y $V2$ pertenecen a W

CONEXO



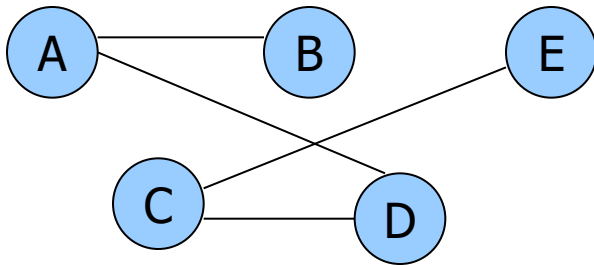
Componentes conexas:
entre ellas son conexas



ALGORITMO: DETERMINAR CONEXIÓN

- Si entre todos los vértices, hay camino
 - Un recorrido desde cualquier vértice
 - Visitara a TODOS los vértices del grafo
- Si no
 - Tendremos una componente conexa
 - *Conjunto de vértices recorrido*
 - Para descubrir otras
 - *Repetir recorrido desde un vértice no visitado*
 - *Hasta que todos los vértices hayan sido visitados*

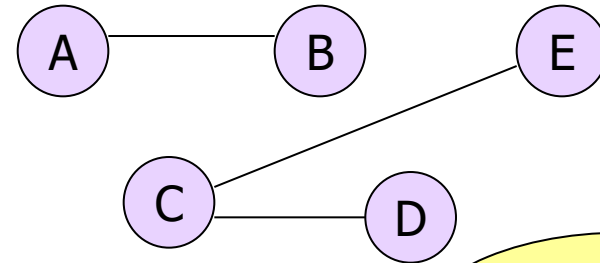
EJEMPLO



Recorrido desde E

E C D A B

Conjunto recorridos =
Conjunto de Vertices
Es CONEXO



Recorrido desde E

E C D

Component
e Conexa
W1

Recorrido desde B

B A

Component
e Conexa
W2

IMPLEMENTACION

No olvidar: Luego de recorrer, obtendremos un conjunto de vertices LRecorrido

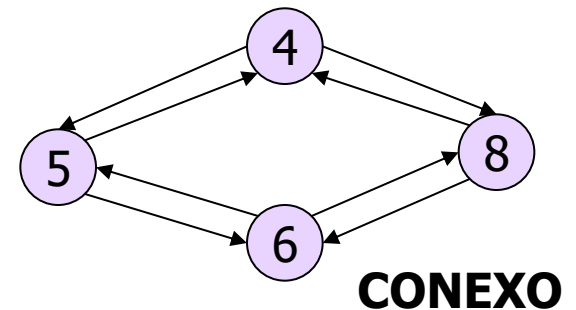
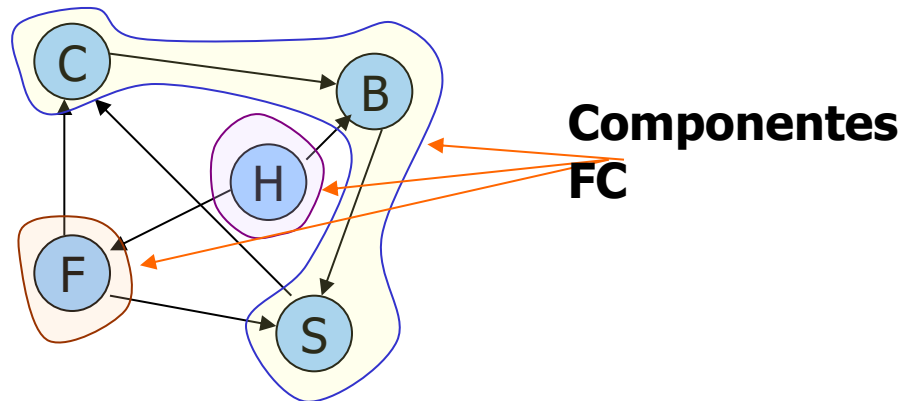
```
boolean getConnectedComponents (List<List<Node>> componentes){  
    Node n;  
    Lista recorrido;  
    while(true){  
        n = g.buscarVerticeNoVisitado();  
        if(n == null)  
            break;  
        recorrido = g.recorrerAnchura(n);  
        componentes.insertLast (recorrido);  
    }  
    if(componentes.size() == 1) return true;  
    return false;  
}
```

Las compon
forman una l
Listas

COMPONENTES FUERTEMENTE CONEXAS

- De un grafo DIRIGIDO
 - Se puede saber si es *FUERTEMENTE CONEXO*
- Si no lo es se pueden conocer sus
 - Componentes Fuertemente Conexas
 - Conjunto W , de vertices del grafo
 - En el cual hay camino desde cualquier $V1$ a cualquier $V2$
 - Donde $V1$ y $V2$ pertenecen a W

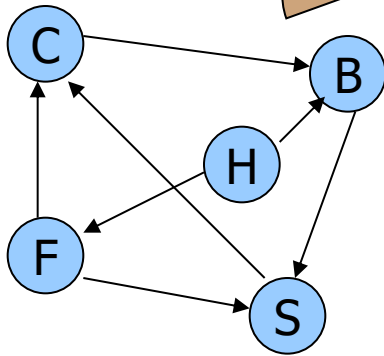
**No
CONEXO**



ALGORITMO

- Dado un V_0 , se desea conocer
 - Los vertices a los que se puede llegar (D)
 - Los vertices que llegan a V_0 (A)
- Si $D \cap A = V$ entonces
 - Hay camino entre cualquier par de vertices
 - Fuertemente conexo
- Si no
 - Tendremos una componente conexa
 - *Conjunto de vertices recorrido*
 - Para descubrir otras
 - *Repetir recorrido desde vertice que no sea elemento de una C.F.C.*
 - *Hasta que todos los vertices esten en C.F.C*

EJEMPLO



$$W = D \cap A$$

$$W1 = \{B, C, S\}$$

$W \leftrightarrow V$,
Componente F.C.

1) Recorrer desde B
(Descendientes)

D = B S C

2) Invertir Direcciones

3) Recorrer desde B
(Ascendientes)

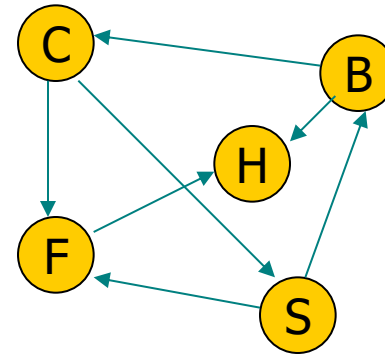
A = B C F H S

1) Recorrer desde H
(Descendientes)

D = H F C B S

3) Recorrer desde H
(Ascendientes)

A = H



$$W2 = \{H\}$$

$$W3 = \{F\}$$

1) Recorrer desde F
(Descendientes)

D = F C B S

3) Recorrer desde F
(Ascendientes)

A = F H